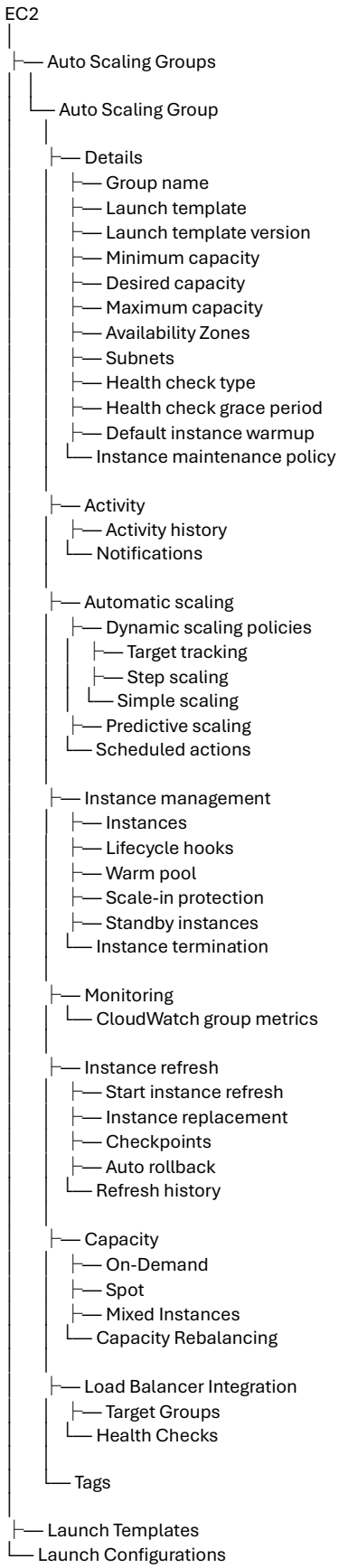
Amazon EC2 Auto Scaling Group — Console Dashboard Wise Interview Questions & Answers

AWS Console Structure



AWS documents that an ASG uses a launch template/configuration plus capacity, Availability Zones/subnets, health checks and scaling policies; launch templates are the modern approach for new configurations. **A** AWS Documentation+1

---

1. What is Amazon EC2 Auto Scaling?
Answer:
Amazon EC2 Auto Scaling automatically manages the number of EC2 instances running for an application.
It can launch new instances when demand increases and terminate instances when demand decreases.
It also replaces unhealthy instances to maintain the required capacity.

We define minimum, desired, and maximum capacity for the group. **A** AWS Documentation+1
For example:
Suppose our application normally needs 2 EC2 instances but during high traffic it needs 5.
Auto Scaling can automatically increase the instances from 2 to 5.
When traffic goes down, it can reduce them again.
Example:
Users
↓
Load Balancer
↓
Auto Scaling Group
↓
EC2 EC2 EC2
Simple ga gurthupettuko:
ASG = Automatically manage EC2 instance count

---

2. What is an Auto Scaling Group?
Answer:
An Auto Scaling Group is a logical group of EC2 instances managed together.
It maintains the desired number of instances and can automatically scale them up or down.
It can also replace unhealthy instances.

The group is controlled using minimum, desired, and maximum capacity settings. **A** AWS Documentation
For example:
Suppose our web application needs at least 2 EC2 instances.
We create an ASG with minimum 2, desired 2, and maximum 6.
ASG maintains the required number and can increase up to 6 when needed.
Example:
Auto Scaling Group
↓
┌──┼──┐
EC2-1 EC2-2 EC2-3
Simple ga gurthupettuko:
ASG = Group of EC2 instances managed together

---

3. Where do we find Auto Scaling Groups in AWS Console?
Answer:
Auto Scaling Groups are available inside the Amazon EC2 console.
From the EC2 navigation pane, we select **Auto Scaling Groups**.
From there we can create, update, monitor, and delete Auto Scaling Groups.

The selected ASG opens its detailed configuration and management tabs. **A** AWS Documentation+1
For example:
Suppose we want to check how many instances our application currently has.
We open EC2 Console → Auto Scaling Groups and select our ASG.
Then we can check its capacity, instances, activities, and scaling policies.
Example:
AWS Console
↓
EC2
↓
Auto Scaling Groups
↓

My-ASG
Simple ga gurthupettuko:
EC2 → Auto Scaling Groups

---

4. What is a Launch Template in an Auto Scaling Group?
Answer:
A Launch Template defines how EC2 instances should be launched.
It can contain the AMI, instance type, key pair, security groups, storage, network settings, and tags.
The ASG uses this configuration whenever it launches new instances.

Launch Templates are the recommended modern way to define instance configuration for ASGs. **A** AWS Documentation+1
For example:
Suppose every web server needs Amazon Linux, t3.micro, a security group, and an IAM role.
We put these settings in a Launch Template.
The ASG uses that template whenever it creates a new web server.
Example:
Launch Template
↓
AMI + Instance Type
+ Security Group + Storage
↓
Auto Scaling Group
↓
EC2 Instances
Simple ga gurthupettuko:
Launch Template = EC2 launch configuration

---

5. What is the Launch Template Version?
Answer:
A Launch Template can have multiple versions of its configuration.
An Auto Scaling Group can use a specific version, the default version, or the latest version.
This allows us to change instance configuration without creating a completely new template.

It is especially useful when updating AMIs or other instance settings. **A** AWS Documentation
For example:
Suppose version 1 uses an old AMI and version 2 uses a new AMI.
We can update the ASG to use version 2 for newly launched instances.
Existing instances are not automatically changed just because the template version changes.
Example:
Launch Template
├— Version 1 → Old AMI
└— Version 2 → New AMI
↓
ASG
Simple ga gurthupettuko:
Template Version = Different versions of EC2 configuration

---

6. What is Minimum Capacity?
Answer:
Minimum capacity defines the smallest number of EC2 instances that the ASG should maintain.
The ASG normally does not scale below this value.
It provides a minimum level of application capacity.
We should choose it based on the application's minimum availability requirement.
For example:
Suppose our website needs at least 2 servers to remain available.
We set minimum capacity to 2.
Even when traffic is low, the ASG maintains at least that required capacity.
Example:
Minimum = 2

ASG
↓
EC2-1
EC2-2
Simple ga gurthupettuko:
Minimum = Lowest instance count

7. What is Desired Capacity?
Answer:
Desired capacity is the number of instances that the ASG tries to maintain at a given time.
When the group is created, it normally launches instances up to the desired capacity.
Scaling policies can change desired capacity within the configured limits.

It is the current target number of instances for the group. **A** AWS Documentation
For example:
Suppose minimum is 2, desired is 3, and maximum is 6.
ASG initially tries to run 3 instances.
If scaling increases desired capacity to 5, ASG launches more instances.
Example:
Min = 2
Desired = 3
Max = 6

ASG → 3 EC2
Simple ga gurthupettuko:
Desired = Current target count

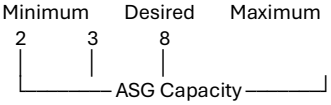
---

8. What is Maximum Capacity?
Answer:
Maximum capacity defines the highest number of EC2 instances the ASG can normally launch.
Scaling policies cannot increase the desired capacity beyond this maximum.
It helps control both application scaling and infrastructure cost.
We choose it based on the maximum expected workload.
For example:
Suppose our application can handle up to 10 servers.
We set maximum capacity to 10.
Even if traffic increases further, the ASG will not normally scale beyond that limit.
Example:
Min = 2
Desired = 4
Max = 10

2 ≤ ASG ≤ 10
Simple ga gurthupettuko:
Maximum = Highest instance count

---

9. What is the difference between Minimum, Desired, and Maximum Capacity?
Answer:
Minimum is the lowest number of instances the ASG should maintain.
Desired is the target number of instances currently required.
Maximum is the highest number of instances the ASG can normally scale to.
Scaling happens between the minimum and maximum limits.
For example:
Suppose Min = 2, Desired = 3, and Max = 8.
The ASG starts with 3 instances.
During high traffic it can increase to 8, but normally it will not go below 2.
Example:



Simple ga gurthupettuko:
Min = Lowest | Desired = Target | Max = Highest

---

10. What are Availability Zones in an ASG?
Answer:
Availability Zones are separate locations inside an AWS Region where ASG instances can run.
We can select multiple Availability Zones to improve application availability.
The ASG can distribute instances across the selected zones.

If one zone has a problem, instances in another zone can continue serving traffic. **A** AWS Documentation
For example:
Suppose our application runs in two Availability Zones.

We configure the ASG with subnets from both zones.
The ASG can maintain instances across both zones instead of depending on only one zone.
Example:
Region
├— AZ-1 → EC2
└— AZ-2 → EC2
↑
ASG
Simple ga gurthupettuko:
Multiple AZs = Better availability

---

11. What is a Subnet in an Auto Scaling Group?
Answer:
The subnets selected in an ASG determine where the EC2 instances can be launched.
We normally select subnets from multiple Availability Zones.
The subnets must belong to the VPC where the instances should run.
ASG uses these subnets when launching and distributing instances.
For example:
Suppose our VPC has private subnets in AZ-1 and AZ-2.
We select both subnets while configuring the ASG.
New EC2 instances can then be launched into those selected subnets.
Example:
VPC
├— Private Subnet-A → AZ-1
└— Private Subnet-B → AZ-2
↑
ASG
Simple ga gurthupettuko:
Subnets = Where ASG launches instances

---

12. What is Health Check Type in an ASG?
Answer:
Health checks allow Auto Scaling to determine whether an instance is healthy.
The common health check types are EC2 and Elastic Load Balancing.
With ELB health checks, the ASG can consider the application's load-balancer health status.

If an instance is unhealthy, ASG can terminate and replace it. **A**  AWS Documentation
For example:
Suppose an EC2 instance is running but the application is not responding.
An ALB health check can detect that the application is unhealthy.
ASG can then replace that unhealthy instance.
Example:
ASG
↓
Health Check
↓
Unhealthy EC2
↓
Terminate
↓
Launch Replacement
Simple ga gurthupettuko:
Health Check = Is my instance/application healthy?

---

13. What is Health Check Grace Period?
Answer:
Health Check Grace Period gives a newly launched instance time to start and become healthy.
During this period, ASG does not immediately replace the instance because of an initial health-check failure.
This is useful when applications take time to boot and initialize.
The value should be long enough for the application to become ready.
For example:
Suppose our application needs 3 minutes to boot.
If the grace period is too short, ASG may think the new instance is unhealthy and replace it.
We can set a suitable grace period so the application gets enough startup time.
Example:
New EC2
↓
Startup
↓

Grace Period
↓
Application Ready
↓
Health Check
Simple ga gurthupettuko:
Grace Period = Give new instance time to start

---

14. What is Default Instance Warmup?
Answer:
Default instance warmup defines how long a newly launched instance needs to warm up before it is considered ready for scaling calculations.
It is particularly useful with dynamic scaling policies.
It prevents newly launched instances from immediately influencing scaling metrics.



This helps scaling decisions become more stable. **A** AWS Documentation
For example:
Suppose a new EC2 instance needs 5 minutes before it can handle normal traffic.
We can configure a 5-minute warmup period.
ASG can then avoid making another scaling decision too quickly.
Example:
New EC2
↓
Warmup = 5 min
↓
Application Ready
↓
Scaling Metrics
Simple ga gurthupettuko:
Warmup = Time for new instance to become ready

---

15. What is Instance Maintenance Policy?
Answer:
Instance maintenance policy controls how instances are replaced during activities such as instance refresh.
It helps balance availability and replacement speed.
The policy can control how much capacity can be unavailable or how much extra capacity can be used during replacement.
It is useful when updating instances without causing unnecessary capacity loss.
For example:
Suppose we want to replace old instances with new instances.
We can configure the maintenance behavior so new instances are launched before old ones are terminated.
This helps maintain application capacity during the replacement.
Example:
Old EC2
↓
New EC2 launched
↓
Healthy
↓
Old EC2 removed
Simple ga gurthupettuko:
Maintenance Policy = How instances are replaced

---

16. What is Activity History in an ASG?
Answer:
Activity History shows scaling activities performed by the Auto Scaling Group.
It can show instance launches, terminations, replacements, and other scaling events.
It is useful for understanding why the number of instances changed.



It is also very useful for troubleshooting failed scaling activities. **A** AWS Documentation
For example:
Suppose our ASG suddenly launches two new EC2 instances.
We can open Activity History and check what scaling activity caused those instances to launch.
This helps us understand and troubleshoot the scaling event.
Example:
ASG
↓
Activity History

├— Launch Instance
├— Terminate Instance
└— Replace Unhealthy Instance

Simple ga gurthupettuko:
Activity History = What ASG did

---

17. What are ASG Notifications?
Answer:
ASG notifications can inform us when important Auto Scaling events occur.
They are commonly delivered using Amazon SNS.
Events can include instance launch, termination, failure, and other lifecycle activities.
This allows administrators or applications to react to ASG events.
For example:
Suppose an instance is terminated because it becomes unhealthy.
ASG can send a notification to an SNS topic.
Administrators or subscribed systems can then receive information about the event.
Example:
ASG Event
↓
SNS
↓
Notification
↓
Admin / Application
Simple ga gurthupettuko:
Notification = Inform when ASG event happens

---

18. What is Automatic Scaling in an ASG?
Answer:
Automatic scaling allows the ASG to automatically increase or decrease instance capacity.
It uses scaling policies, schedules, or predictive scaling information.
The ASG changes desired capacity based on configured conditions.

This helps match EC2 capacity with application demand. **A** AWS Documentation+1
For example:
Suppose CPU usage becomes very high because many users are accessing the application.
A scaling policy can increase the desired capacity.
ASG then launches additional EC2 instances.
Example:
High Traffic
↓
Scaling Policy
↓
Desired Capacity ↑
↓
New EC2 Instances
Simple ga gurthupettuko:
Automatic Scaling = Automatically change instance count

---

19. What is Dynamic Scaling?
Answer:
Dynamic scaling changes ASG capacity based on current application or CloudWatch metrics.
It can respond to changing workload conditions automatically.
Common dynamic scaling approaches include target tracking and step scaling.
It is useful for applications where traffic changes throughout the day.
For example:
Suppose our application normally uses 40% CPU but traffic increases CPU to 70%.
A dynamic scaling policy can detect the condition and launch additional instances.
When demand falls, it can reduce the capacity.
Example:
CPU ↑
↓
Dynamic Scaling
↓
EC2 Count ↑

CPU ↓
↓
Dynamic Scaling

↓
EC2 Count ↓
Simple ga gurthupettuko:
Dynamic = Scale based on current conditions

---

20. What is Target Tracking Scaling?
Answer:
Target tracking scaling tries to keep a selected metric close to a target value.
We choose a metric such as average CPU utilization and define a target percentage.
ASG automatically adjusts the desired capacity to maintain that target.

It is one of the simplest scaling methods to configure. **A**  AWS Documentation
For example:
Suppose we set the target CPU utilization to 50%.
If CPU becomes much higher than 50%, ASG launches more instances.
If CPU stays lower, ASG can reduce instances when appropriate.
Example:
Target CPU = 50%

CPU 80%
↓
Scale Out

CPU 30%
↓
Scale In
Simple ga gurthupettuko:
Target Tracking = Keep metric near target

---

21. What is Step Scaling?
Answer:
Step scaling changes capacity by different amounts depending on how much a metric crosses a threshold.
Instead of always adding the same number of instances, different steps can have different adjustments.
This is useful when a small problem needs a small scale-out but a large spike needs a bigger scale-out.
The scaling behavior is defined using CloudWatch alarm thresholds.
For example:
Suppose CPU reaches 60%, so we add 1 instance.
If CPU reaches 80%, we add 2 instances.
If CPU reaches 95%, we may add 4 instances.
Example:
CPU 60% → +1 EC2
CPU 80% → +2 EC2
CPU 95% → +4 EC2
Simple ga gurthupettuko:
Step Scaling = Different metric levels → Different scaling actions

---

22. What is Simple Scaling?
Answer:
Simple scaling uses a CloudWatch alarm to trigger a scaling action.
When the alarm condition occurs, the ASG adds or removes a configured number of instances.
After a scaling activity, a cooldown period can help avoid repeated immediate scaling actions.
It is simpler but generally less flexible than newer scaling approaches.
For example:
Suppose CPU goes above 70%.
The alarm triggers a simple scaling policy that adds two instances.
The ASG then waits according to the configured behavior before another scaling action.
Example:
CPU > 70%
↓
CloudWatch Alarm
↓
Simple Scaling
↓
+2 EC2
Simple ga gurthupettuko:
Simple Scaling = Alarm → Fixed scaling action

---

23. What is Predictive Scaling?

Answer:
Predictive scaling uses historical workload patterns to forecast future demand.
It can schedule capacity increases before expected traffic occurs.
This is useful for workloads that have predictable patterns.
It can work together with dynamic scaling for both forecasted and real-time demand.
For example:
Suppose an online shopping application gets high traffic every evening.
Predictive scaling can learn the pattern and prepare additional capacity before the evening traffic starts.
Dynamic scaling can then respond to unexpected traffic changes.
Example:
Historical Data
↓
Prediction
↓
Expected Traffic ↑
↓
EC2 Capacity ↑
Simple ga gurthupettuko:
Predictive = Scale before expected demand

---

24. What are Scheduled Actions?
Answer:
Scheduled actions change ASG capacity at specific times.
They are useful when we already know when traffic will increase or decrease.
We can define a schedule that changes minimum, maximum, or desired capacity.
This is useful for predictable business workloads.
For example:
Suppose our application gets heavy traffic every weekday from 9 AM to 6 PM.
We can schedule more instances before 9 AM.
After business hours, we can reduce capacity.
Example:
8:30 AM
↓
Desired = 6

6:30 PM
↓
Desired = 2
Simple ga gurthupettuko:
Scheduled = Scale at a specific time

---

25. What is Instance Management?
Answer:
Instance Management is the area where we manage EC2 instances belonging to the ASG.
It shows instances currently running in the group and their status.
It also provides controls such as standby, lifecycle hooks, and scale-in protection.
This section is useful when managing individual ASG instances.
For example:
Suppose one EC2 instance needs maintenance.
We can inspect that instance from Instance Management and temporarily place it into standby if required.
The ASG can then manage the remaining capacity based on the configuration.
Example:
ASG
↓
Instance Management
├— EC2-1
├— EC2-2
└— EC2-3
Simple ga gurthupettuko:
Instance Management = Manage ASG's EC2 instances

---

26. What is Scale-In Protection?
Answer:
Scale-in protection prevents selected instances from being terminated during scale-in activities.
It is useful when certain instances need to complete important work before termination.
The protection can be enabled for individual instances.
The ASG can still scale other instances according to its scaling behavior.
For example:
Suppose one EC2 instance is processing an important long-running task.
We enable scale-in protection for that instance.
During scale-in, ASG avoids selecting that protected instance for termination.

Example:
ASG
├— EC2-1 → Protected
├— EC2-2
└— EC2-3

Scale In
↓
EC2-2 / EC2-3
Simple ga gurthupettuko:
Scale-In Protection = Protect instance from scale-in termination

---

27. What is Standby Instance?
Answer:
A standby instance is temporarily removed from active service while remaining part of the ASG.
It can be useful when we need to perform maintenance or troubleshooting on an instance.
The instance is not normally used to serve traffic while in standby.
We can later return it to service.
For example:
Suppose EC2-2 has a configuration problem and needs maintenance.
We put it into Standby so it is temporarily out of service.
After maintenance, we can return the instance to service.
Example:
ASG
↓
EC2-1 → InService
EC2-2 → Standby
EC2-3 → InService
Simple ga gurthupettuko:
Standby = Temporarily remove instance from service

---

28. What is a Lifecycle Hook?
Answer:
A lifecycle hook pauses an instance during an Auto Scaling lifecycle transition.
It gives us time to perform custom actions before the instance enters service or before it terminates.
The instance remains in a wait state until the lifecycle action is completed or times out.

It is useful for initialization, cleanup, and event-driven workflows. **A** AWS Documentation+1
For example:
Suppose a new EC2 instance needs to download configuration files before receiving traffic.
A lifecycle hook can pause the instance while the setup is completed.
After setup finishes, the instance continues into service.
Example:
New EC2
↓
Lifecycle Hook
↓
Setup / Custom Action
↓
Complete
↓
InService
Simple ga gurthupettuko:
Lifecycle Hook = Pause instance for custom action

---

29. What are Instance Launch and Termination Lifecycle Hooks?
Answer:
Lifecycle hooks can be configured for instance launch or instance termination events.
A launch hook allows custom work before a new instance enters service.
A termination hook allows cleanup or state-saving work before an instance is terminated.

Separate hooks can be used for launch and termination behavior. **A** AWS Documentation
For example:
During launch, we may download application configuration.
During termination, we may save logs or perform cleanup.
Lifecycle hooks provide time for these custom actions.
Example:

Launch
EC2 → Hook → Setup → InService

Terminate
EC2 → Hook → Cleanup → Terminated
Simple ga gurthupettuko:
Launch Hook = Setup | Termination Hook = Cleanup

---

30. What is Heartbeat Timeout in a Lifecycle Hook?
Answer:
Heartbeat timeout defines how long an instance can remain in the lifecycle wait state before the lifecycle action times out.
We can send heartbeat signals to extend the wait when the custom action needs more time.
If the action is completed, we complete the lifecycle action.



This prevents instances from remaining paused indefinitely. **A** AWS Documentation
For example:
Suppose an instance needs 5 minutes to complete a startup task.
We configure a suitable heartbeat timeout and send a completion signal when the task finishes.
The instance then continues to the next lifecycle state.
Example:
EC2
↓
Lifecycle Hook
↓
Wait
↓
Heartbeat / Complete
↓
Next State
Simple ga gurthupettuko:
Heartbeat = Tell ASG custom action is still running

---

31. What is a Warm Pool?
Answer:
A Warm Pool keeps pre-initialized EC2 instances ready for future scaling activities.
Instead of starting a completely new instance from the beginning, ASG can use a warm instance when more capacity is needed.
This can reduce application startup time.



It is especially useful for applications with long boot or initialization times. **A** AWS Documentation
For example:
Suppose our application takes 10 minutes to start on a new EC2 instance.
We can keep some initialized instances in a warm pool.
When traffic increases, those instances can become active faster.
Example:
Warm Pool
 ├— EC2
 └— EC2
  ↓
Traffic ↑
  ↓
ASG
  ↓
Move to InService
Simple ga gurthupettuko:
Warm Pool = Ready instances for faster scale-out

---

32. What is ASG Monitoring?
Answer:
ASG monitoring provides metrics about the Auto Scaling Group.
CloudWatch group metrics can show information such as desired, in-service, pending, and terminating instances.
These metrics help us understand the current capacity and scaling behavior.



They are also useful for troubleshooting scaling problems. **A** AWS Documentation+1
For example:
Suppose our ASG desired capacity is 5 but only 3 instances are in service.

We can use monitoring metrics and activity history to investigate why the other instances are not available.
This helps identify launch or health-check problems.
Example:
ASG
↓
CloudWatch
├— Desired
├— InService
├— Pending
└— Terminating
Simple ga gurthupettuko:
Monitoring = Watch ASG capacity and behavior

---

33. What is CloudWatch Group Metrics Collection?
Answer:
CloudWatch group metrics collection allows Auto Scaling to publish metrics for the ASG.
These metrics provide visibility into the number and state of instances in the group.
They can be used for monitoring and troubleshooting.

We can enable this option from the ASG configuration. **A** AWS Documentation+1
For example:
Suppose we want to see how many instances are currently pending or terminating.
We enable group metrics collection and monitor those values in CloudWatch.
This gives us a better view of ASG behavior.
Example:
ASG
↓
Group Metrics
↓
CloudWatch
↓
Dashboard / Alarms
Simple ga gurthupettuko:
Group Metrics = ASG metrics in CloudWatch

---

34. What is Instance Refresh?
Answer:
Instance Refresh is used to replace existing ASG instances with new instances based on updated configuration.
It is commonly used after changing an AMI or Launch Template version.
Instances can be replaced gradually instead of terminating everything at once.

This makes rolling updates easier and safer. **A** AWS Documentation+1
For example:
Suppose our ASG has 10 instances using an old AMI.
We update the Launch Template with a new AMI and start Instance Refresh.
ASG gradually replaces old instances with new ones.
Example:
Old EC2-1
Old EC2-2
Old EC2-3
↓
Instance Refresh
↓
New EC2-1
New EC2-2
New EC2-3
Simple ga gurthupettuko:
Instance Refresh = Replace old instances with new configuration

---

35. What is Instance Refresh Replacement Method?
Answer:
The replacement method defines how existing instances are replaced during Instance Refresh.
The replacement can be performed by terminating old instances and launching replacements according to the configured availability behavior.

Instance maintenance settings can influence this behavior.

The goal is to update instances while maintaining the required application capacity. **A**  AWS Documentation
For example:
Suppose we have 6 instances and need to update their AMI.
ASG replaces instances gradually according to the refresh and maintenance configuration.
This avoids unnecessarily taking the entire application offline.
Example:
Old Instance
↓
Replace
↓
New Instance
↓
Healthy
Simple ga gurthupettuko:
Replacement Method = How old instances become new instances

---

36. What are Instance Refresh Checkpoints?
Answer:
Checkpoints allow an Instance Refresh to pause after replacing a portion of the instances.
This can be useful when we want to verify that the new configuration works before continuing.
It supports a controlled rollout of a new AMI or Launch Template version.
This is useful for safer production deployments.
For example:
Suppose we have 100 instances.
We can update a small portion first and verify application health.
If everything works correctly, the refresh can continue with more instances.
Example:
100 EC2
↓
10% Refresh
↓
Checkpoint
↓
Verify
↓
Continue
Simple ga gurthupettuko:
Checkpoint = Pause refresh and verify

---

37. What is Auto Rollback in Instance Refresh?
Answer:
Auto rollback allows an Instance Refresh to automatically return to the previous configuration when the refresh fails.
It can help reduce the impact of a bad AMI or incorrect instance configuration.
Rollback is useful when a new version causes application problems.

The feature has prerequisites that must be met before it can be used. **A**  AWS Documentation
For example:
Suppose we deploy a new AMI but the application fails health checks.
If auto rollback is configured and the refresh fails, ASG can roll back according to the supported rollback behavior.
This helps recover from a failed instance update.
Example:
Old Version
↓
New Version
↓
Health Check Failed
↓
Auto Rollback
↓
Previous Version
Simple ga gurthupettuko:
Auto Rollback = Failed refresh → Go back

---

38. What is Instance Refresh History?

Answer:
Instance Refresh History shows previous and current refresh operations for an ASG.
It provides the status of refresh activities such as Pending, InProgress, and Successful.
It helps us understand when a refresh started and whether it completed successfully.

Activity History can provide additional details about individual launches and terminations. **A** AWS Documentation
For example:
Suppose we updated our application last night using Instance Refresh.
Today we can open refresh history and verify whether the update completed successfully.
If there was a problem, activity history can help investigate it.
Example:
Instance Refresh History
├— Pending
├— InProgress
└— Successful
Simple ga gurthupettuko:
Refresh History = Instance update history

---

39. What is a Mixed Instances Policy?
Answer:
A Mixed Instances Policy allows an ASG to use multiple EC2 instance types and purchase options.
It can combine On-Demand and Spot Instances.
This provides more flexibility in capacity and can help optimize cost.

Mixed instances are configured using a Launch Template with ASG settings. **A** AWS Documentation+1
For example:
Suppose our application can run on several compatible instance types.
We can configure the ASG to use multiple instance types and combine Spot with On-Demand capacity.
This gives the ASG more options when launching capacity.
Example:
ASG
├— On-Demand
│ └— Instance Type A
│
└— Spot
├— Instance Type B
└— Instance Type C
Simple ga gurthupettuko:
Mixed Instances = Multiple instance types + purchase options

---

40. What is On-Demand Capacity in an ASG?
Answer:
On-Demand capacity uses EC2 On-Demand Instances without relying on Spot pricing availability.
These instances provide more predictable capacity compared with Spot Instances.
An ASG can be configured to maintain a specified amount of On-Demand capacity.
This is useful when application capacity needs stronger availability guarantees.
For example:
Suppose our production application requires a reliable base capacity.
We can keep some instances as On-Demand and use Spot for additional flexible capacity.
This provides a balance between reliability and cost.
Example:
ASG
↓
Base Capacity
↓
On-Demand EC2
Simple ga gurthupettuko:
On-Demand = More predictable EC2 capacity

---

41. What is Spot Capacity in an ASG?
Answer:
Spot capacity uses unused EC2 capacity at discounted prices.
Spot Instances can be interrupted by AWS when the capacity is needed back.

The ASG can attempt to replace interrupted Spot Instances to maintain desired capacity.

Spot is useful for workloads that can tolerate interruptions. **A**  AWS Documentation
For example:
Suppose our application has background processing jobs that can tolerate interruptions.
We can run those workloads using Spot Instances through the ASG.
This can significantly reduce compute cost.
Example:
ASG
↓
Spot Instances
↓
Low Cost
↓
Workload
Simple ga gurthupettuko:
Spot = Lower-cost but interruptible capacity

---

42. What is Capacity Rebalancing?
Answer:
Capacity Rebalancing helps an ASG respond when Spot Instances are at increased risk of interruption.
The ASG can proactively launch replacement capacity before an at-risk Spot Instance is interrupted.
This helps maintain application capacity when using Spot Instances.
It is especially useful for workloads using mixed Spot and On-Demand capacity.
For example:
Suppose AWS indicates that one Spot Instance may soon be interrupted.
Capacity Rebalancing can start another instance before the original instance disappears.
This reduces the chance of losing application capacity.
Example:
Spot EC2
↓
Interruption Risk
↓
Capacity Rebalancing
↓
New EC2
↓
Replace Risky Instance
Simple ga gurthupettuko:
Capacity Rebalancing = Replace risky Spot capacity early

---

43. What is Load Balancer Integration with an ASG?
Answer:
An ASG can be associated with an Elastic Load Balancer target group.
When instances launch, they can automatically be registered with the target group.
When instances terminate, they are removed from the target group.
This allows the load balancer to distribute application traffic across healthy ASG instances.
For example:
Suppose our ASG launches a new EC2 instance.
The instance is automatically registered with the ALB target group.
The ALB can then send traffic to that instance after it becomes healthy.
Example:
Users
↓
ALB
↓
Target Group
↓
ASG
↓
EC2 EC2 EC2
Simple ga gurthupettuko:
ASG + ALB = Automatically manage application servers

---

44. How does an ALB Health Check work with an ASG?
Answer:
An ALB health check checks whether an application on an EC2 instance is responding correctly.
The ALB marks instances healthy or unhealthy based on the configured health-check path and conditions.

When ASG uses ELB health checks, it can use this health status when replacing instances.

This helps ensure traffic goes to healthy application instances. **A**  AWS Documentation

For example:

Suppose the ALB checks /health on every EC2 instance.

If one instance stops responding correctly, the ALB marks it unhealthy.

ASG can then replace that instance when ELB health checking is configured.

Example:

ALB

↓

/health

↓

EC2

↓

Healthy / Unhealthy

↓

ASG Replacement

Simple ga gurthupettuko:

ELB Health Check = Check application health

---

45. What are ASG Tags?

Answer:

Tags are key-value labels attached to Auto Scaling Groups and, when configured, their launched instances.

They help identify applications, environments, teams, and cost ownership.

We can use tags such as Environment=Production or Application=Web.

Proper tag propagation makes it easier to manage and track EC2 resources.

For example:

Suppose we have production and development ASGs.

We can tag them with their environment names.

The instances launched by the ASG can also receive the configured tags.

Example:

ASG

↓

Tags

├— Environment = Production

├— Application = Web

└— Team = DevOps

Simple ga gurthupettuko:

Tags = Identify and organize resources

---

46. What is Termination Policy in an ASG?

Answer:

Termination Policy determines which instance the ASG should terminate when scaling in.

AWS provides different termination policy behaviors to help choose an instance for termination.

The selection can consider factors such as Availability Zone balance and instance age depending on the policy.

It is useful when we want more control over scale-in behavior.

For example:

Suppose our ASG has 6 instances and needs to reduce to 4.

The termination policy helps ASG decide which two instances should be removed.

This helps control how scale-in occurs.

Example:

ASG

├— EC2-1

├— EC2-2

├— EC2-3

├— EC2-4

├— EC2-5

└— EC2-6

↓

Termination Policy

↓

Remove selected instances

Simple ga gurthupettuko:

Termination Policy = Which instance should scale-in remove?

---

47. What is Availability Zone Rebalancing in an ASG?

Answer:

When an ASG uses multiple Availability Zones, it tries to keep capacity balanced across the selected zones.

If the distribution becomes unbalanced, Auto Scaling can launch or terminate instances to restore balance.

This helps maintain capacity across Availability Zones.

It improves resilience against a problem affecting one Availability Zone. **A** AWS Documentation
For example:
Suppose we have 4 instances across two AZs.
If one AZ temporarily has fewer instances, ASG can rebalance the capacity.
This keeps the application more evenly distributed.
Example:
Before:
AZ-1 → 3 EC2
AZ-2 → 1 EC2

↓ Rebalance

After:
AZ-1 → 2 EC2
AZ-2 → 2 EC2
Simple ga gurthupettuko:
AZ Rebalancing = Keep instances balanced across AZs

---

48. What is a Cooldown Period in Auto Scaling?
Answer:
A cooldown period is a period intended to give a scaling action time to take effect before another scaling decision is made.
It was commonly associated with simple scaling policies.
The purpose is to reduce rapid repeated scaling actions.
Modern target tracking and other approaches use instance warmup and related mechanisms for better control.
For example:
Suppose ASG launches 2 instances because CPU is high.
We do not want another immediate scaling action before those instances become useful.
A suitable cooldown or warmup approach gives the new instances time to start.
Example:
Scale Out
↓
New EC2
↓
Wait
↓
Observe Metrics
↓
Next Scaling Decision
Simple ga gurthupettuko:
Cooldown = Give scaling action time to settle

---

49. What happens when an ASG instance becomes unhealthy?
Answer:
Auto Scaling continuously checks the health of instances in the group.
If an instance is determined to be unhealthy, ASG can terminate it.
It then launches a replacement to maintain the desired capacity.

This is one of the core high-availability features of Auto Scaling. **A** AWS Documentation+1
For example:
Suppose our ASG desired capacity is 3 and one EC2 instance fails.
ASG detects the unhealthy instance and terminates it.
Then ASG launches a new instance so the group returns to 3.
Example:
ASG = 3
↓
EC2-2 Unhealthy
↓
Terminate
↓
Launch New EC2
↓
ASG = 3
Simple ga gurthupettuko:
Unhealthy Instance = Replace automatically

---

50. What happens when an ASG instance is manually terminated?
Answer:
If an instance belonging to an ASG is terminated, the ASG detects that actual capacity has fallen below desired capacity.
It normally launches a replacement instance to restore the desired capacity.
This is why we should not think of ASG instances as permanent individual servers.



The ASG manages the overall capacity rather than preserving a specific instance. **A** AWS Documentation+1
For example:
Suppose desired capacity is 3 and we manually terminate one EC2 instance.
The ASG sees that only 2 instances remain.
It launches another instance to return the group to 3.
Example:
Desired = 3

EC2-1
EC2-2
EC2-3
↓
EC2-2 Terminated
↓
ASG launches EC2-4
↓
3 Instances
Simple ga gurthupettuko:
Manual termination → ASG replaces it

---

51. What is the relationship between ASG and AMI?
Answer:
The AMI defines the operating system and base software used when launching an EC2 instance.
The AMI is normally referenced by the Launch Template used by the ASG.
When ASG launches a new instance, EC2 uses that AMI to create the instance.
Updating the AMI usually requires updating the Launch Template and then refreshing instances.
For example:
Suppose our old instances use AMI version 1.
We create a new AMI with security updates and create a new Launch Template version.
Then we can use Instance Refresh to replace the old instances.
Example:
New AMI
↓
Launch Template v2
↓
ASG
↓
Instance Refresh
↓
New EC2 Instances
Simple ga gurthupettuko:
AMI = Base image used by ASG-launched EC2

---

52. What is the difference between Launch Template and ASG?
Answer:
A Launch Template defines **how an EC2 instance should be created**.
An Auto Scaling Group defines **how many instances should run and how they should be managed**.
The ASG uses the Launch Template when it needs to launch instances.
They work together but solve different problems.
For example:
Launch Template says use Amazon Linux, t3.micro, and this security group.
ASG says keep 2 instances running and scale up to 6.
Together they create and manage the application's EC2 fleet.
Example:
Launch Template
= How to create EC2

ASG
= How many EC2 + when to scale
Simple ga gurthupettuko:
Launch Template = How | ASG = How many/manage

---

53. What is Instance Protection vs Lifecycle Hook?

Answer:
Instance scale-in protection prevents an instance from being selected for termination during scale-in.
A lifecycle hook instead pauses an instance during a lifecycle event so custom actions can be performed.
Protection is mainly about preventing scale-in termination.
Lifecycle hooks are mainly about controlling what happens during launch or termination.
For example:
If an instance is processing an important task, scale-in protection can prevent it from being terminated.
If a new instance needs setup before serving traffic, a lifecycle hook can pause it during launch.
So the two features solve different problems.
Example:
Scale-In Protection
↓
Don't terminate instance

Lifecycle Hook
↓
Pause instance
↓
Perform custom action
Simple ga gurthupettuko:
Protection = Don't terminate | Hook = Pause and act

---

54. What is ASG with multiple instance types?
Answer:
An ASG can be configured to use multiple compatible EC2 instance types through a mixed instances configuration.
This gives Auto Scaling more choices when trying to obtain capacity.
It can also combine different purchase options such as On-Demand and Spot.

This can improve capacity flexibility and cost optimization. **A** AWS Documentation
For example:
Suppose our application works with t3.large and t3a.large.
We can allow both types in the ASG.
If one type has limited capacity, ASG has another compatible option.
Example:
ASG
├── t3.large
├── t3a.large
└── Spot / On-Demand
Simple ga gurthupettuko:
Multiple Types = More EC2 choices

---

55. What is an ASG launch failure?
Answer:
A launch failure happens when the ASG cannot successfully create a required EC2 instance.
Possible causes include an invalid Launch Template, unavailable instance capacity, networking problems, security configuration issues, or insufficient permissions.
Activity History is one of the first places to check for the failure reason.
We should also check the Launch Template, subnet, security group, IAM role, and instance availability.
For example:
Suppose desired capacity is 4 but only 3 instances are running.
Activity History shows that the fourth instance failed to launch.
We investigate the reported reason and correct the configuration.
Example:
ASG
↓
Launch EC2
↓
FAILED
↓
Activity History
↓
Find Reason
↓
Fix Configuration
Simple ga gurthupettuko:
Launch Failure = Check Activity History first

---

56. How do you update an ASG with a new AMI?

Answer:
First, we create or update a Launch Template version using the new AMI.
Then we configure the ASG to use the required Launch Template version.
Existing instances are not automatically replaced just because the template changes.

We can use Instance Refresh to gradually replace the old instances. **A** AWS Documentation+1
For example:
Suppose our ASG currently uses AMI version 1.
We create Launch Template version 2 with AMI version 2.
Then we start Instance Refresh so existing instances are gradually replaced.
Example:
Old AMI
↓
Launch Template v1
↓
ASG

New AMI
↓
Launch Template v2
↓
Instance Refresh
↓
New EC2
Simple ga gurthupettuko:
New AMI = New LT Version + Instance Refresh

---

57. How does ASG provide High Availability?
Answer:
ASG improves availability by maintaining the required number of EC2 instances.
We can distribute instances across multiple Availability Zones.
If an instance becomes unhealthy, ASG can replace it automatically.
When combined with an ALB, traffic can be distributed to healthy instances.
For example:
Suppose we have 4 web servers across two Availability Zones.
If one server fails, ASG launches a replacement.
The ALB continues sending traffic to the healthy servers.
Example:
```
        ALB
         ↓
        ASG
      /     \
    AZ-1     AZ-2
   EC2 EC2   EC2 EC2
```

Failure → Replacement
Simple ga gurthupettuko:
ASG + Multi-AZ + ALB = High Availability

---

58. How does ASG handle sudden high traffic?
Answer:
When traffic increases, CloudWatch metrics can indicate increased resource usage.
A configured scaling policy can then increase the ASG desired capacity.
ASG launches additional EC2 instances using the Launch Template.
If those instances pass health checks, the load balancer can start sending traffic to them.
For example:
Suppose CPU increases from 40% to 80% because many users access the website.
A target tracking policy detects the increase and ASG launches additional instances.
The new healthy instances then start handling application traffic.
Example:
Traffic ↑
↓
CPU ↑
↓
CloudWatch
↓
Scaling Policy
↓
ASG

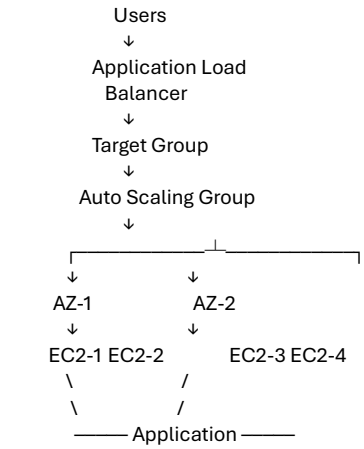
↓
New EC2
Simple ga gurthupettuko:
High Traffic = Scale Out

---

59. How does ASG handle low traffic?
Answer:
When application demand decreases, scaling metrics can indicate that fewer instances are required.
A scaling policy can reduce the desired capacity within the configured minimum and maximum limits.
ASG then terminates selected instances according to its scaling and termination behavior.
This reduces unnecessary EC2 usage and cost.
For example:
Suppose our application has 8 instances during peak traffic.
At night, traffic becomes very low and the scaling policy reduces desired capacity to 3.
ASG terminates unnecessary instances and keeps the required capacity.
Example:
Traffic ↓
↓
Metric ↓
↓
Scaling Policy
↓
Desired Capacity ↓
↓
EC2 Count ↓
Simple ga gurthupettuko:
Low Traffic = Scale In

---

60. Explain a complete real-world Auto Scaling Group architecture.
Answer:
Suppose we have a web application running on EC2 behind an Application Load Balancer.
We create a Launch Template containing the AMI, instance type, security group, IAM role, and storage settings.
We create an ASG across multiple Availability Zones with minimum, desired, and maximum capacity.
We use health checks and scaling policies so ASG replaces unhealthy instances and adds or removes capacity based on demand.
For example:
Suppose our normal traffic needs 2 EC2 instances and peak traffic needs 6.
We configure Min = 2, Desired = 2, Max = 6 and use target tracking for CPU utilization.
When traffic increases, ASG launches new instances and the ALB sends traffic to them.
Example:



Launch Template
↓
AMI + Instance Type
+ Security Group + IAM + Storage

Auto Scaling
↓
Min = 2
Desired = 2
Max = 6

High Traffic
↓
CloudWatch Metric

↓
Target Tracking
↓
Scale Out
↓
New EC2 Instances

Low Traffic
↓
Target Tracking
↓
Scale In
↓
Remove Extra EC2

Simple ga gurthupettuko:
ASG = Launch Template + Capacity + Health Checks + Scaling + Multi-AZ + Instance Management + Instance Refresh

---